

PROJECT TITLE : EVENTFORCE MANAGEMENT SYSTEM

**COLLEGE NAME : ARASU ENGINEERING COLLEGE,
KUMBAKONAM.**

**TEAM LEADER
(WITH EMAIL ID) : SHAMEEM AKTHAR S
(shameemakthar1105@gmail.com)**

**TEAM MEMBER 1
(WITH EMAIL ID) : KISHORE KUMAR S
(saikishorek321@gmail.com)**

**TEAM MEMBER 2
(WITH EMAIL ID) : PARAMESHWARAN K
(parameshkrishnabalan@gmail.com)**

**TEAM MEMBER 3
(WITH EMAIL ID) : ABINATH S
(abinaths05@gmail.com)**

**TEAM MEMBER 4
(WITH EMAIL ID) : PRAVEEN A
(praveenayyadurai2311@gmail.com)**

EVENTFORCE MANAGEMENT SYSTEM

SALESFORCE IMPLEMENTATION

User Story:

The EventForce Management System is a centralized Salesforce solution for running event-planning operations: client bookings, event scheduling, venue reservations, vendor coordination, cancellation approvals, reminders, and feedback collection. Event teams work with Events, Clients, Vendors, Venues, and Feedback records that are connected through a structured data model. Flows send event reminders and update process status, while an Approval Process controls cancellation requests. Apex classes and triggers handle venue availability and block double bookings. A Lightning App, reports, and a dashboard give the team a live view of schedules, vendor activity, and operational performance. Together these features improve booking coordination, data accuracy, communication, and the client experience.

1. INTRODUCTION

1.1 Project Overview

EventForce is a Salesforce CRM application built for Event Planner Solutions, a business that previously tracked events, clients, venues, and vendors in separate places. The application brings all of this into one platform. It models the business with six custom objects (Event, Client, Vendor, Venue, Feedback, and the EventVendor junction object) and automates the routine work with validation rules, formula fields, record-triggered flows, an approval process, Apex triggers and classes, and a nightly batch job.

Users reach everything through a custom Lightning App named Event Planner. Access is controlled by profiles, roles, a permission set, organization-wide defaults, and sharing rules, so each group of users sees only what it needs. The whole solution was configured and tested in a Salesforce Developer Edition org for academic purposes, following the practices of a real implementation.

1.2 Purpose

The system exists to make event planning faster, more accurate, and easier to monitor. Specifically, it is designed to:

- Keep client, event, venue, vendor, and feedback information in one place.
- Simplify event booking and venue reservation.
- Prevent the same venue from being booked twice on the same date.
- Send timely reminders to clients and route cancellations through an approval step.
- Improve coordination between clients, event coordinators, vendor managers, and venue managers.
- Provide reports and a dashboard that support day-to-day decisions.

1.3 Objectives and Business Value

The main objective is to digitize and automate event management for Event Planner Solutions, giving the team a single, reliable view of every event from enquiry to feedback. The business value comes from:

- **Less manual work:** reminders, status changes, and cancellation handling run automatically.
- **Fewer booking conflicts:** Apex logic checks the venue and date before an event is saved.
- **Better communication:** clients receive an automatic reminder three days before a confirmed event.
- **Controlled decisions:** cancellations need approval, and access to data follows each user's role.
- **Visibility for management:** reports and the EventForce Operations Dashboard show upcoming events and budgets at a glance.

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Event Operations Manager	manage event planning operations efficiently	client, venue, and vendor information is hard to manage centrally	event activities need coordination across several areas	frustrated and concerned
PS-2	Event Coordinator	create and manage event bookings	event, client, and venue details need many manual steps	there is no single platform for the whole event lifecycle	overwhelmed and frustrated
PS-3	Venue Manager	manage venue availability accurately	one venue may be assigned to several events	bookings need availability tracking and double-booking prevention	concerned and uncertain
PS-4	Vendor Manager	coordinate vendors for different events	vendor details and assignments are managed separately	an event can need several vendors and services	overwhelmed and concerned
PS-5	Event Coordinator	send event information and reminders on time	reminders are sent by hand	clients must be notified before upcoming events	stressed and concerned

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-6	Event Administrator	manage cancellation requests systematically	requests need approval and status updates	cancellation decisions need a controlled process	frustrated and uncertain
PS-7	Event Coordinator	collect and manage client feedback	feedback must be kept with event and client details	feedback relates to both the event and the client	concerned about customer satisfaction
PS-8	Event Operations Manager	monitor event operations and performance	operational information is not visible in one place	event data must be turned into reports and dashboards	concerned and less confident in decisions

2.2 Empathy Map Canvas

User Persona: Event Coordinator

Primary responsibility: managing bookings, client requirements, venue reservations, vendor coordination, reminders, and event status.

Quadrant	Description
Think & Feel	Wants events to run smoothly, avoid booking conflicts, and give clients a dependable experience.
Hear	Requirements and updates from clients, vendors, venue managers, and administrators.
See	Event details, client records, venue availability, vendor assignments, cancellations, and feedback.
Say & Do	Coordinates with stakeholders, updates event records, manages bookings, and follows up on requirements.
Pain	Manual coordination, scattered information, the risk of double-booking a venue, and difficulty tracking many events.
Gain	Centralized records, automated reminders and approvals, accurate venue availability, and better visibility.

2.3 Brainstorming and Idea Prioritization

Ideas Considered

Idea	Approach	Key Limitation
Manual registers	Record event, client, venue, and vendor details on paper	Slow and hard to maintain

Idea	Approach	Key Limitation
Spreadsheet tracking	Track bookings and events in spreadsheets	Little automation and no real-time coordination
Standalone portal	Build a dedicated event-management portal	High development and maintenance effort
Salesforce CRM	Centralize event operations on the Salesforce platform	Needs careful configuration and implementation

Final Selection

Salesforce CRM was selected for three reasons:

- **Automation** – reminders, approvals, status updates, and other processes run without manual effort.
- **Scalability** – the platform can grow with more users, records, and requirements.
- **Real-time visibility** – centralized data with reports and dashboards supports monitoring.

Decision: Salesforce CRM is the best fit for a centralized, automated, and scalable EventForce Management System.

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stage	Journey Stage	What Happens
1	Event Enquiry and Client Registration	Client requirements and contact details are captured in Salesforce.
2	Event Planning and Booking	Event type, date, budget, and status are created and managed.
3	Venue Selection and Reservation	A suitable venue is chosen, availability is checked, and double bookings are prevented.
4	Vendor Coordination	Vendors and services are assigned to the event and their status is tracked.
5	Event Confirmation and Communication	Confirmed events trigger an automatic reminder to the client.
6	Event Execution and Monitoring	Progress is followed through records, reports, and the dashboard.
7	Cancellation and Approval	Cancellation requests go through the approval process with status updates and notifications.
8	Feedback and Closure	Client feedback is recorded and completed events are updated for tracking and analysis.

3.2 Solution Requirements

Functional Requirements

FR No	Requirement
FR-1	Event management
FR-2	Client management
FR-3	Venue management
FR-4	Vendor management
FR-5	Event–vendor assignment
FR-6	Venue availability and double-booking prevention
FR-7	Automated event reminders
FR-8	Event cancellation approval
FR-9	Feedback management
FR-10	Reports and dashboards
FR-11	Event status automation
FR-12	Data migration through the Data Import Wizard

Non-Functional Requirements

NFR	Description
Usability	A Lightning App makes event management and navigation simple.
Security	Access is controlled through profiles, roles, and permission sets.
Reliability	Automation, approvals, and venue validation behave accurately.
Performance	Event and operational data are processed efficiently.
Availability	Authorized users can reach the cloud-based org at any time.
Scalability	The model supports growing numbers of events, clients, vendors, venues, and users.
Maintainability	Flows, Apex, validation rules, and security settings can be monitored and updated.
Data Integrity	Validation rules and controlled data migration keep records accurate.

3.3 Data Flow Diagram

Main Flow

Client / Event Team → Enter client and event details → Salesforce EventForce system → Validate and store data → Event, Client, Venue, and Vendor records → Check venue

availability → Assign venue and vendors → Confirm event → Automated 3-day reminder → Event execution and status updates → Feedback collection → Reports and dashboards.

Supporting Flows

- **Cancellation request:** Event Coordinator → cancellation request → Approval Process → Approved or Rejected → event status update → notifications.
- **Venue booking validation:** Event saved → Apex trigger → VenueStatusHelper → check existing bookings for venue and date → block double booking or update venue availability.
- **Event completion:** Event date passes → Scheduled Apex → Batch Apex → Event Status set to Completed.

Simple DFD Representation

Clients, Event Coordinators, Vendor Managers, and Venue Managers → Salesforce CRM – EventForce (Lightning App: Event Planner) → Custom objects (Event, Client, Vendor, Venue, Feedback, EventVendor) → Automation and business logic (Flows, Approval Process, Apex) → Operational data and status updates → Reports and dashboards → Event operations and decision-making.

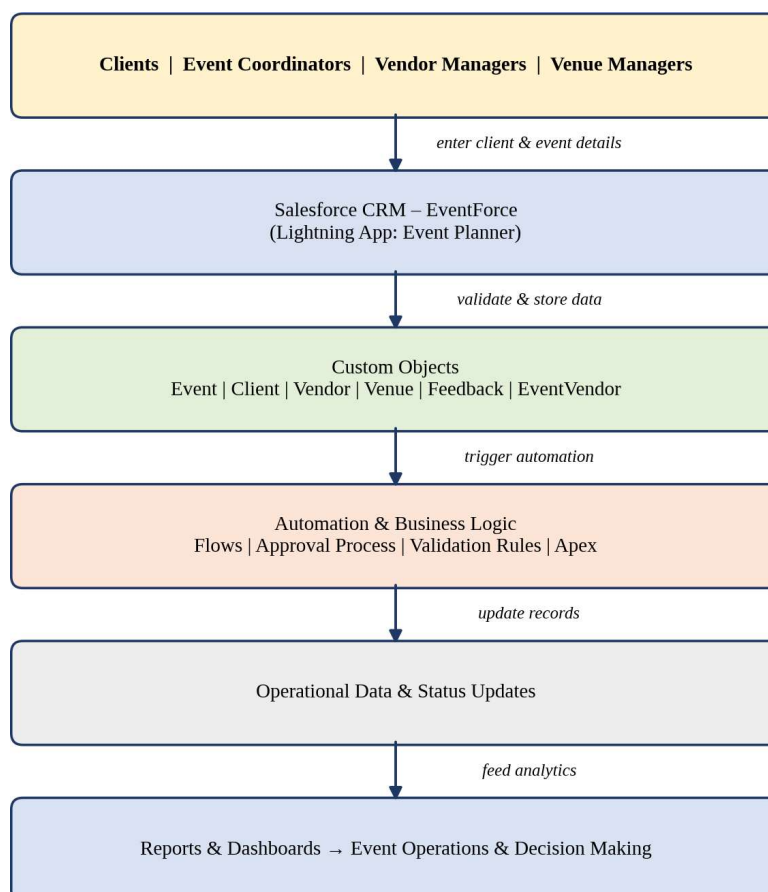


Figure 1: Data flow diagram of the EventForce Management System

3.4 Technology Stack

Layer	Technology
Platform	Salesforce CRM, Developer Edition
User Interface	Salesforce Lightning App
Database	Salesforce custom objects
Business Logic	Flows, formula fields, validation rules
Approval	Salesforce Approval Process
Automation	Record-triggered flows, Apex triggers
Processing	Batch Apex and Scheduled Apex
Security	Profiles, roles, permission sets, sharing rules, organization-wide defaults
Reporting	Reports and dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

Each problem identified in the ideation phase is matched with a Salesforce capability in the table below.

Problem	EventForce Solution	Benefit
Client, event, venue, and vendor data is hard to manage centrally	Custom objects: Event, Client, Venue, Vendor, Feedback, EventVendor	Centralized, structured event data
Event booking needs many manual steps	Flows and automated processes	Less manual effort and faster processing
Risk of venue double booking	PreventDoubleBooking trigger and VenueStatusHelper class	Venue availability is validated and conflicts are blocked
Vendor coordination is hard to track	EventVendor junction object and Vendor records	Several vendors can be assigned to one event
Clients may miss reminders	3-day reminder flow	Automatic reminder before confirmed events
Cancellations need controlled handling	Event_Cancellation_Process approval process	Structured approval with status updates
Feedback must be tied to events	Feedback object with Event and Client relationships	Organized feedback management
Operations are hard to monitor	Reports and the EventForce Operations Dashboard	Better visibility for decisions
Users need different levels of access	Profiles, roles, permission sets, OWD, sharing rules	Controlled and secure access

4.2 Proposed Solution

Parameter	Description
Problem	Event management is manual and fragmented.
Solution	A Salesforce CRM application with automation.
Innovation	Automatic reminders, venue validation, and cancellation approvals.
Impact	Faster coordination and better operational visibility.
Business Model	Centralized event planning and management.
Scalability	Supports growth in events, clients, vendors, venues, and users.

4.3 Solution Architecture

The solution is made up of the following components:

- Custom objects: Event, Client, Vendor, Venue, Feedback, and the EventVendor junction object
- Validation rules and formula fields
- Flows and the Approval Process
- Apex triggers, Apex classes, Batch Apex, and Scheduled Apex
- Profiles, roles, permission sets, and sharing rules
- Data Import Wizard for data migration
- Lightning App, reports, and dashboards

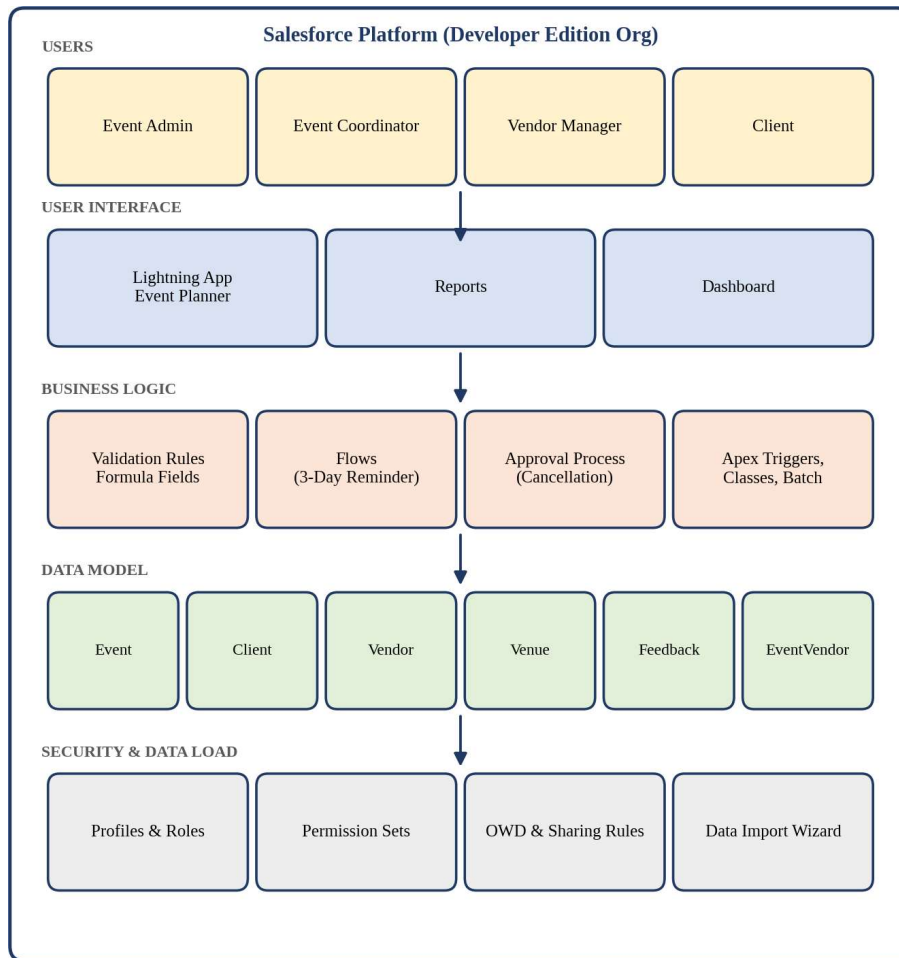


Figure 2: Solution architecture of the EventForce Management System

5. PROJECT PLANNING AND SCHEDULING

5.1 Project Planning

The project followed an Agile approach: requirements were grouped into epics, split into user stories with story points, and delivered in four sprints. Velocity from each sprint was used to estimate the next.

Product Backlog and Sprint Schedule

Sprint	Epic	US No	User Story / Task	Points	Priority	Member
Sprint-1	Event Management	US-01	As an Event Coordinator, I want to create events with type, date, and status so that event information is standardized.	3	High	Member 1
Sprint-1	Client and Venue Management	US-02	As an Event Coordinator, I want to link a client and venue to an event so that coordination is centralized.	5	High	Member 2

Sprint	Epic	US No	User Story / Task	Points	Priority	Member
Sprint-2	Vendor Management	US-03	As a Vendor Manager, I want to assign several vendors to an event so that services can be coordinated.	5	High	Member 3
Sprint-2	Notifications and Automation	US-04	As a Client, I want timely event reminders so that I know when my event is approaching.	3	High	Member 4
Sprint-3	Approval Management	US-05	As an Event Coordinator, I want cancellation requests to follow an approval process so that cancellations are controlled.	5	High	Member 1
Sprint-3	Venue Management and Apex	US-06	As a Venue Manager, I want double bookings prevented so that venue schedules stay accurate.	8	High	Member 2
Sprint-4	Security and Access Control	US-07	As an Event Administrator, I want role-based access so that users see only appropriate information.	5	High	Member 3
Sprint-4	Reports and Dashboards	US-08	As Management, I want reports and dashboards so that event operations can be monitored.	5	Medium	Member 4

Project Tracker, Velocity and Burndown

Sprint	Total Points	Duration	Start Date (Planned)	End Date (Planned)	Points Completed	Release Date (Actual)
Sprint-1	20	6 days	DD Mon YYYY	DD Mon YYYY		
Sprint-2	20	6 days	DD Mon YYYY	DD Mon YYYY		
Sprint-3	20	6 days	DD Mon YYYY	DD Mon YYYY		
Sprint-4	20	6 days	DD Mon YYYY	DD Mon YYYY		

Average velocity = total story points ÷ number of sprints = 80 ÷ 4 = 20 points per sprint.
 Replace the date placeholders with your own sprint dates.

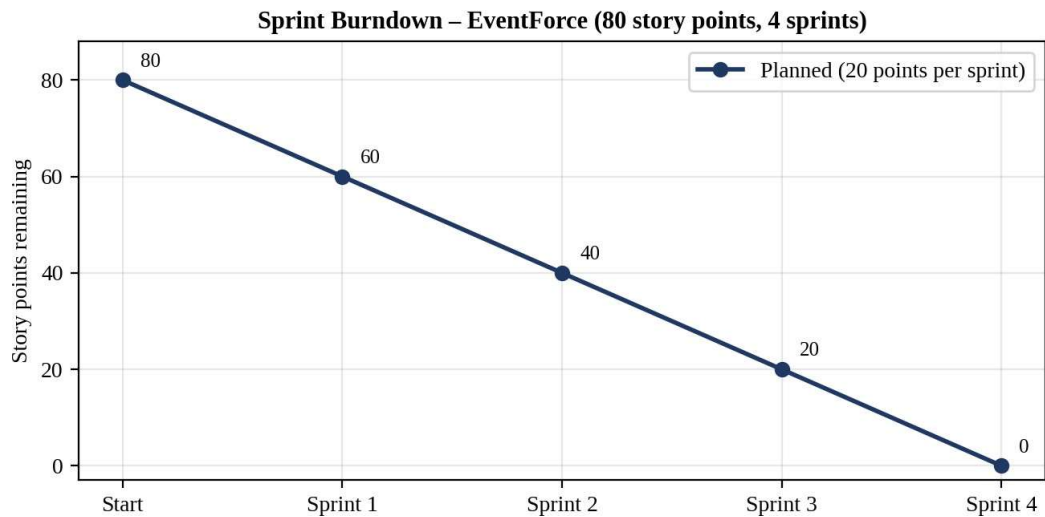


Figure 3: Planned sprint burndown chart

6. PROJECT DEVELOPMENT PHASE

Phase 1: Requirement Analysis and Planning

The goal of this phase was to define what the EventForce Management System must do and how it would be built. The system lets event planners manage client bookings, vendor coordination, venue reservations, and feedback in a single CRM, automates scheduling, cancellation approvals, and client reminders, and reports on operations in real time.

Functional Scope

- Custom objects: Event, Client, Vendor, Venue, Feedback, and EventVendor (junction object).
- Relationships: lookup, master-detail, and many-to-many (Event ↔ Vendor).
- Flows for reminders, feedback notifications, and approval updates.
- Validation rules and formula fields for accurate data entry and budget calculation.
- An approval process for event cancellation, with email alerts to coordinators and clients.
- Profiles, roles, and permission sets for user-level access.
- Reports and dashboards for event summaries and upcoming-event insights.

Stakeholder Mapping

Stakeholder	Role in the System
Clients	Request event services, give feedback, and view updates.
Event Coordinators	Manage scheduling, client communication, and approval workflows.
Vendors	Deliver assigned services such as catering, decor, and logistics.

Stakeholder	Role in the System
Venue Managers	Manage venue availability and reservations.
Event Administrators	Oversee system operations, security, and data monitoring.

Execution Roadmap

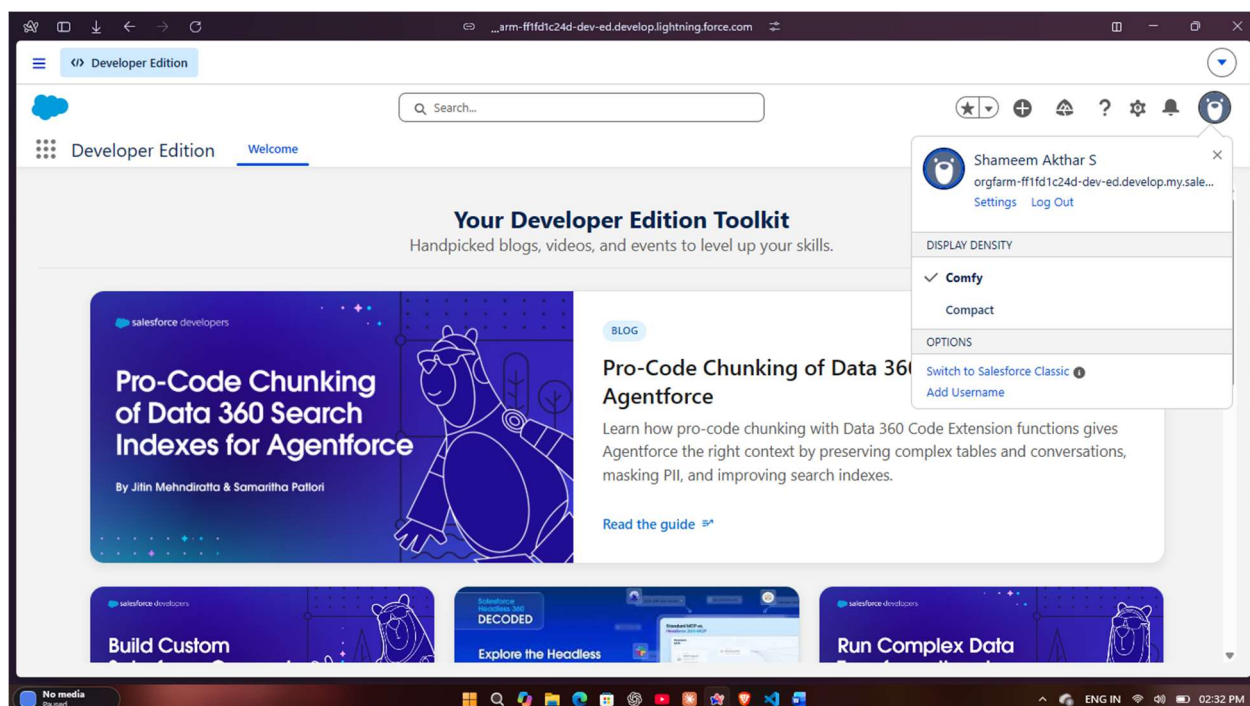
- Object and field setup (Event, Client, Vendor, Venue, Feedback, EventVendor).
- Validation rules and formula fields.
- Profiles, roles, users, and permission sets.
- Approval process for event cancellation.
- Automation flows for event reminders.
- Apex classes and triggers to update venue availability and prevent double bookings.
- Reports and dashboards.

Phase 2: Backend Development and Configurations

This phase built the working core of the system. Declarative tools (validation rules, formula fields, approval processes, and record-triggered flows) automate reminders and cancellations, while Apex handles the logic that declarative tools cannot, namely venue availability and double-booking prevention. Security settings were configured alongside, so the application became a dynamic, automated, and secure CRM.

Milestone 1: Salesforce Developer Account

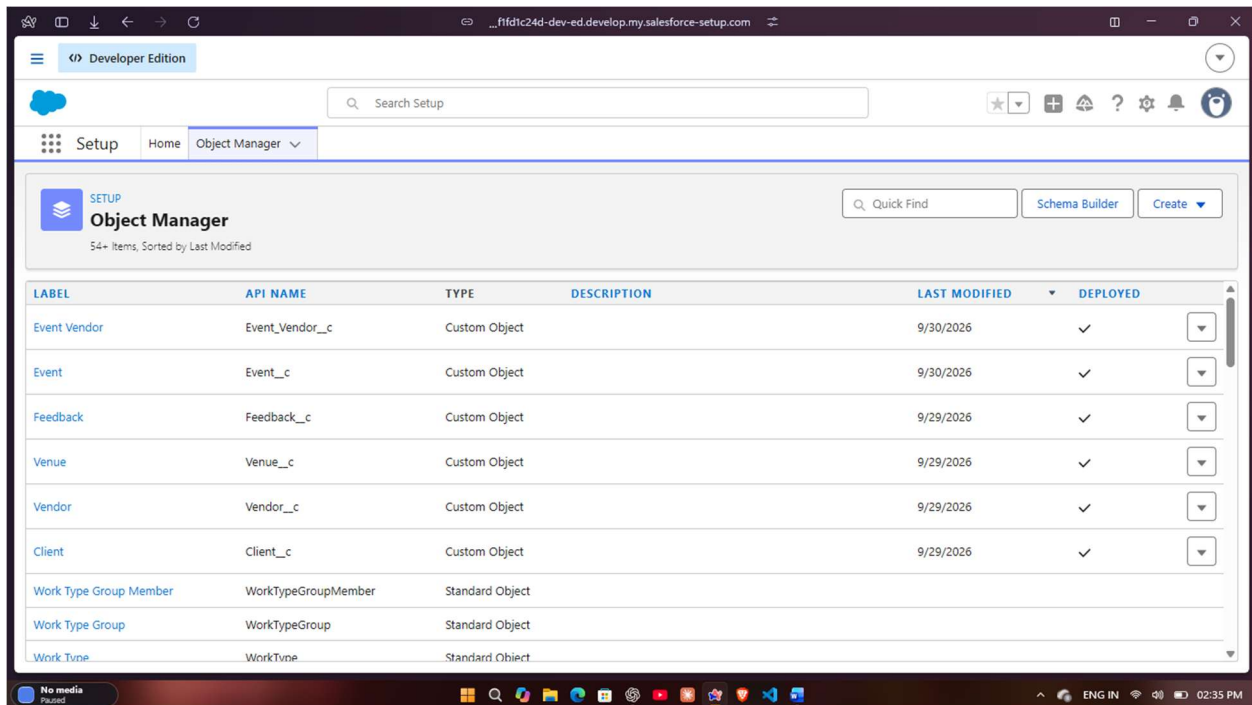
- Open the Salesforce Developer Edition sign-up page and enter your name, email, role (Developer), company (college name), country, postal code, and a unique username in the form name@organization.com.
- Open the verification email, click Verify Account, set a password, and answer the security question to reach the Setup home page.



Milestone 2: Custom Objects

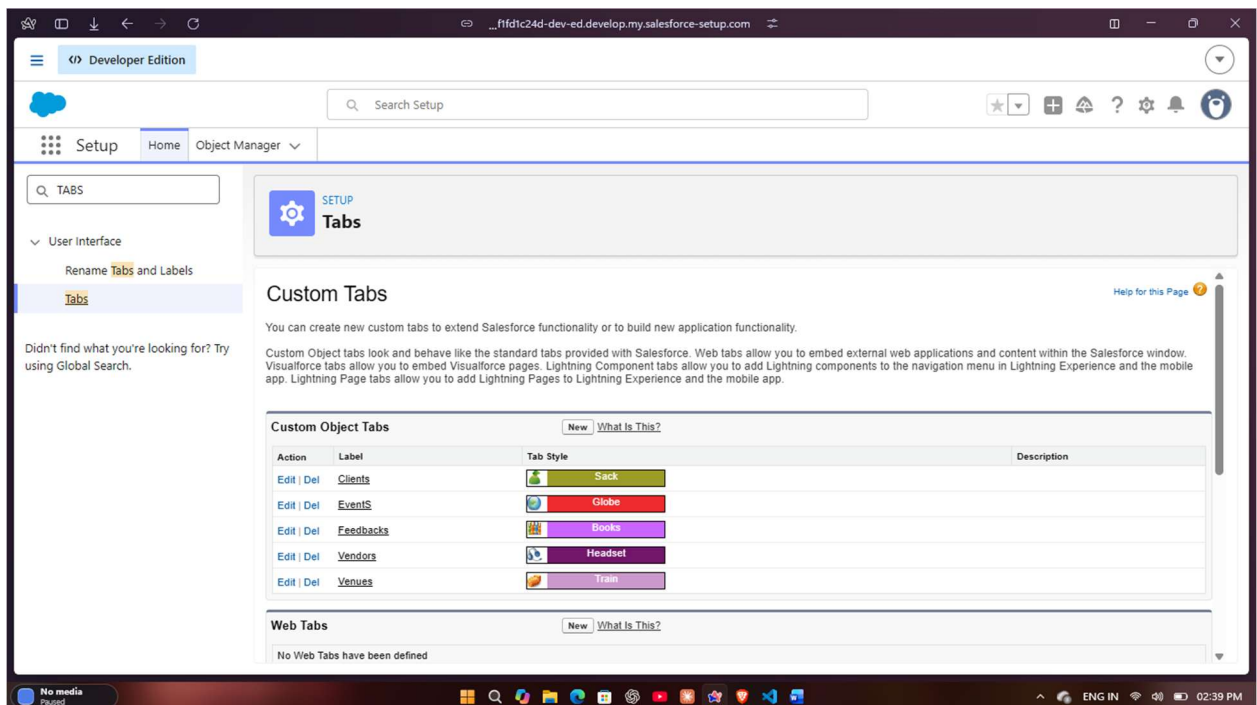
An object is a database table that stores organization-specific data. Each custom object was created from Setup → Object Manager → Create → Custom Object, with Allow Reports, Allow Activities, and Track Field History enabled, Deployment Status set to Deployed, Allow Search turned on, and the Notes and Attachments related list added.

Object	API Name	Key Fields
Event	Event__c	Event Name, Event Date, Event Type, Event Status, Event Budget
Client	Client__c	Client Name, Email, Phone, Address, Country, City
Vendor	Vendor__c	Vendor Name, Email, Phone, Service Type, Status
Venue	Venue__c	Venue Name, Address, Location (URL), Capacity, Availability Status
Feedback	Feedback__c	Feedback ID (Auto Number), Rating, Comments
Event Vendor	EventVendor__c	Junction object linking Event and Vendor



Milestone 3: Tabs

A custom object tab was created for Event, Client, Vendor, Venue, and Feedback (Setup → Tabs → New under Custom Object Tabs), with a tab style selected and the other options left at their defaults.



Milestone 4: Fields and Relationships

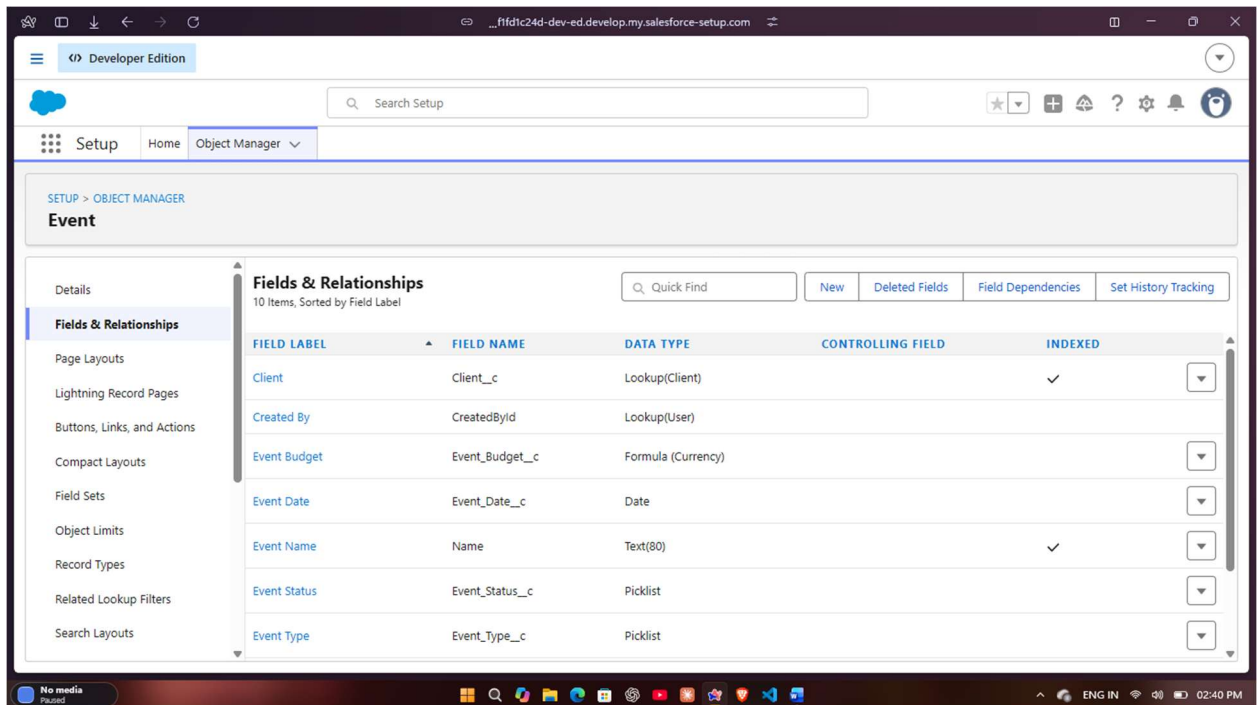
Standard fields exist on every object; custom fields were added to meet the business requirements. The data types chosen for each field are listed below.

Object	Field	Type and Values
Event	Event Type	Picklist: Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event	Event Date	Date
Event	Event Status	Picklist: Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event	Event Budget	Formula (Currency), based on Event Type
Client	Email, Phone, Address	Email, Phone, Text Area
Client	Country, City	Picklists with the countries and cities used by the sample clients
Vendor	Service Type	Multi-select picklist: Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Vendor	Email, Phone, Status	Email, Phone, Picklist (Available, Booked, Cancelled)
Venue	Address, Location, Capacity	Text Area, URL, Number
Venue	Availability Status	Picklist: Available, Reserved, Not Available
Feedback	Rating, Comments	Picklist (1 to 5), Long Text Area

Formula Field: Event Budget

The budget is calculated from the event type:

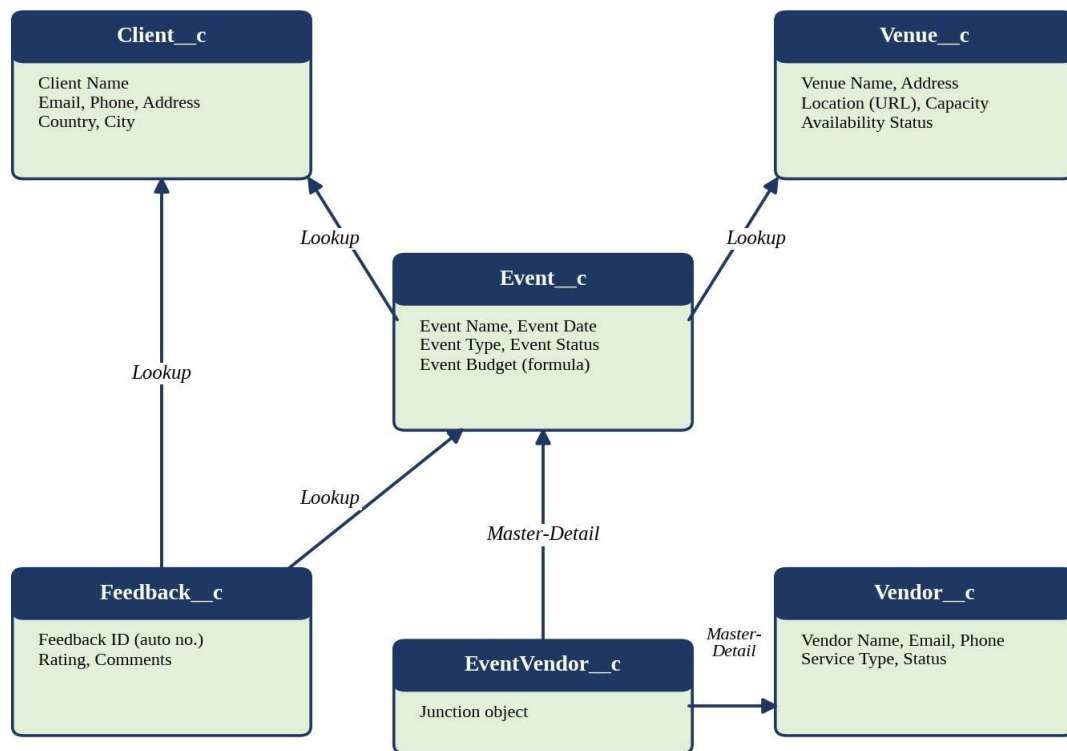
```
CASE (
  TEXT(Event_Type_c),
  "Wedding", 50000,
  "Corporate", 30000,
  "Birthday", 10000,
  "Anniversary", 20000,
  "Festival", 60000,
  "Concert", 40000,
  "Other", 15000,
  0
)
```



Relationships

Relationship	Type
Event → Client	Lookup
Event → Venue	Lookup
Feedback → Event	Lookup
Feedback → Client	Lookup
EventVendor → Event	Master-Detail
EventVendor → Vendor	Master-Detail

Salesforce does not support a direct many-to-many relationship, so the EventVendor junction object holds two master-detail relationships, one to Event and one to Vendor. This lets one event have many vendors and one vendor serve many events.

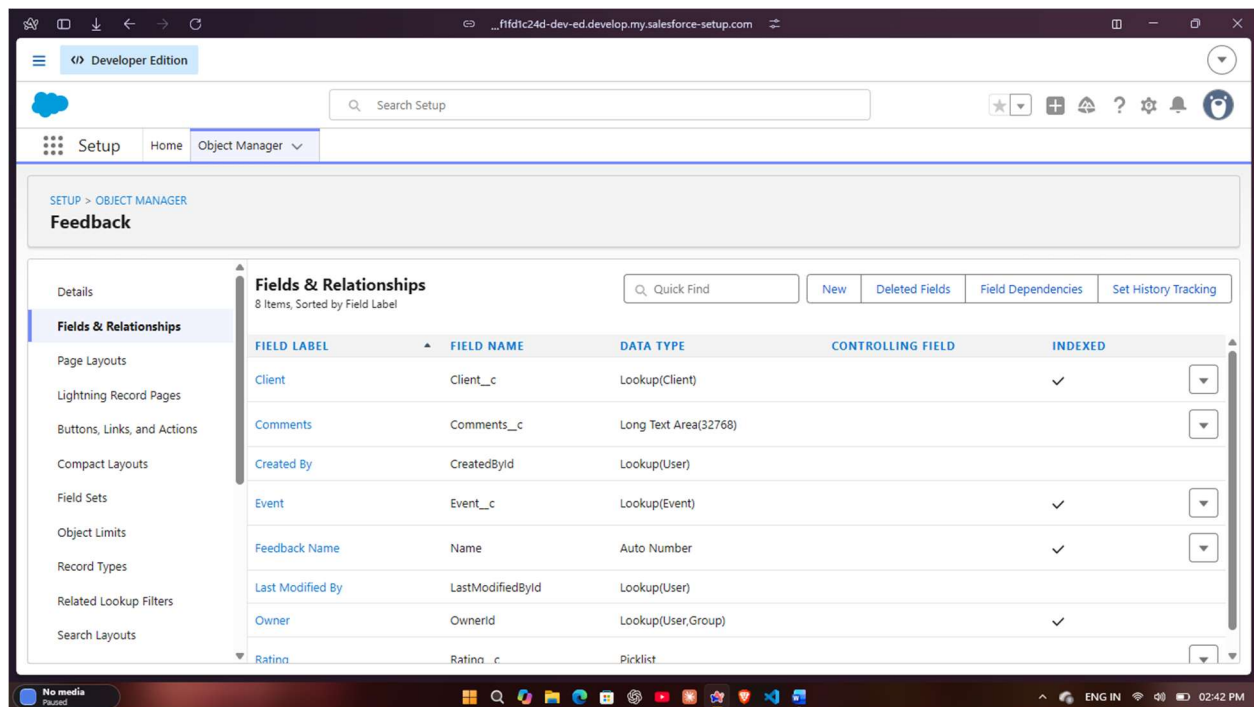


Arrows point from the child object to the parent object it references.

Figure 4: Data model: objects and relationships

Lookup Filter on Feedback

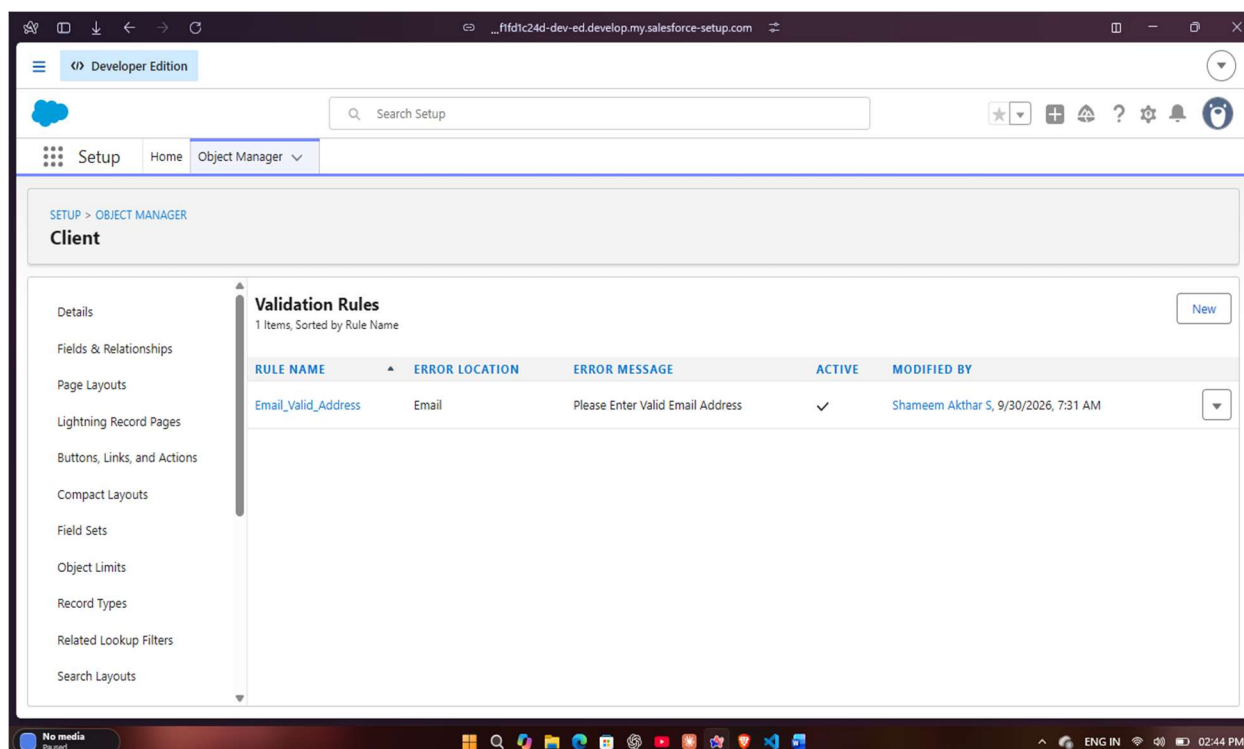
To make sure feedback is linked only to an event that belongs to the same client, a lookup filter was added to the Event field on Feedback: Event: Client equals Feedback: Client (filter enabled).



Milestone 5: Validation Rules

A validation rule checks data when a record is saved and stops the save if the condition is not met. The rule Email_Valid_Address was created on Client__c to confirm that the email address has a valid format.

```
Rule Name: Email_Valid_Address
Active: Yes
Formula: NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9._]+\.[a-zA-Z]{2,}$"))
Message: Please Enter Valid Email Address
Location: Field → Email
```



Milestone 6: Approval Process (Event Cancellation)

Profiles, roles, and users must exist before this process is set up, because approvers and recipients depend on them. Four classic email templates were created first:

Template	Purpose
Event Cancellation Request Notification	Sent to the approver when a cancellation request is submitted.
Email to Client After Cancellation	Tells the client that the event has been canceled.
Approval Notification to Event Owner	Tells the owner the request was approved and the status is now Canceled.
Rejection Notification to Event Owner	Tells the owner the request was rejected and the status returns to Confirmed.

The approval process Event_Cancellation_Process was then configured on the Event object:

Setting	Configuration
Entry criteria	Event Status equals Pending Cancellation
Approver	Manager Review step, assigned to a specific user or the submitter's manager
Initial submitters	Event Owner

Setting	Configuration
Initial submission action	Email alert to the Event Coordinator role
Final approval actions	Field update: Event Status = Canceled; email alert to the Event Owner
Final rejection actions	Field update: Event Status = Rejected; email alert to the Event Owner
Activation	Process activated from its detail page

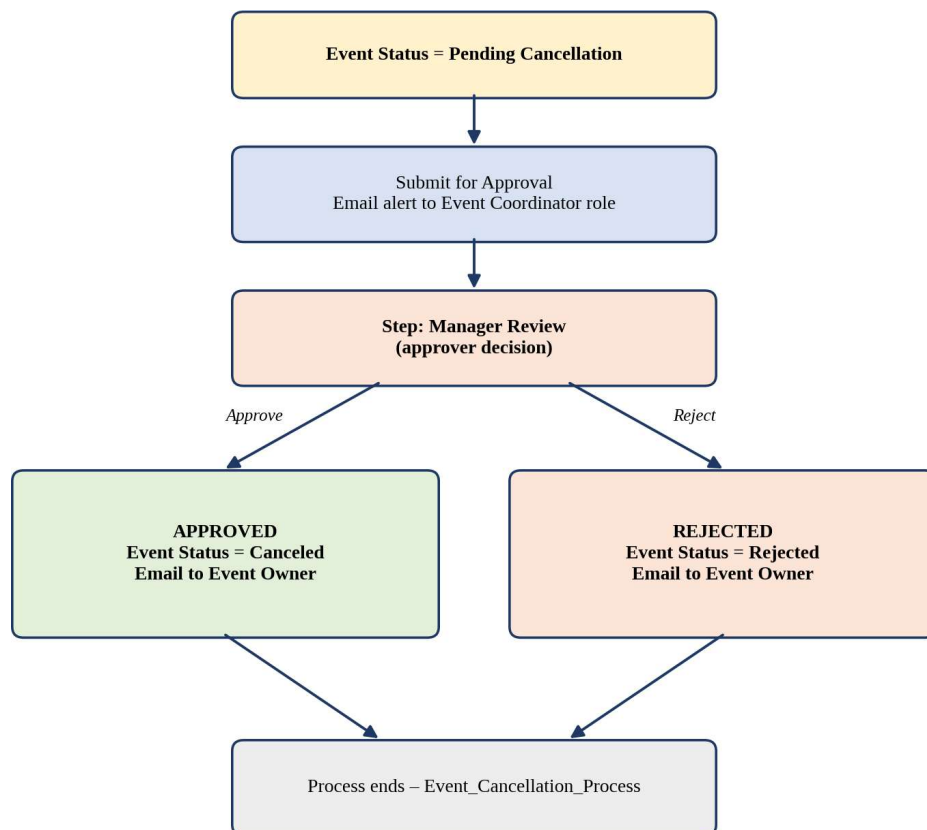
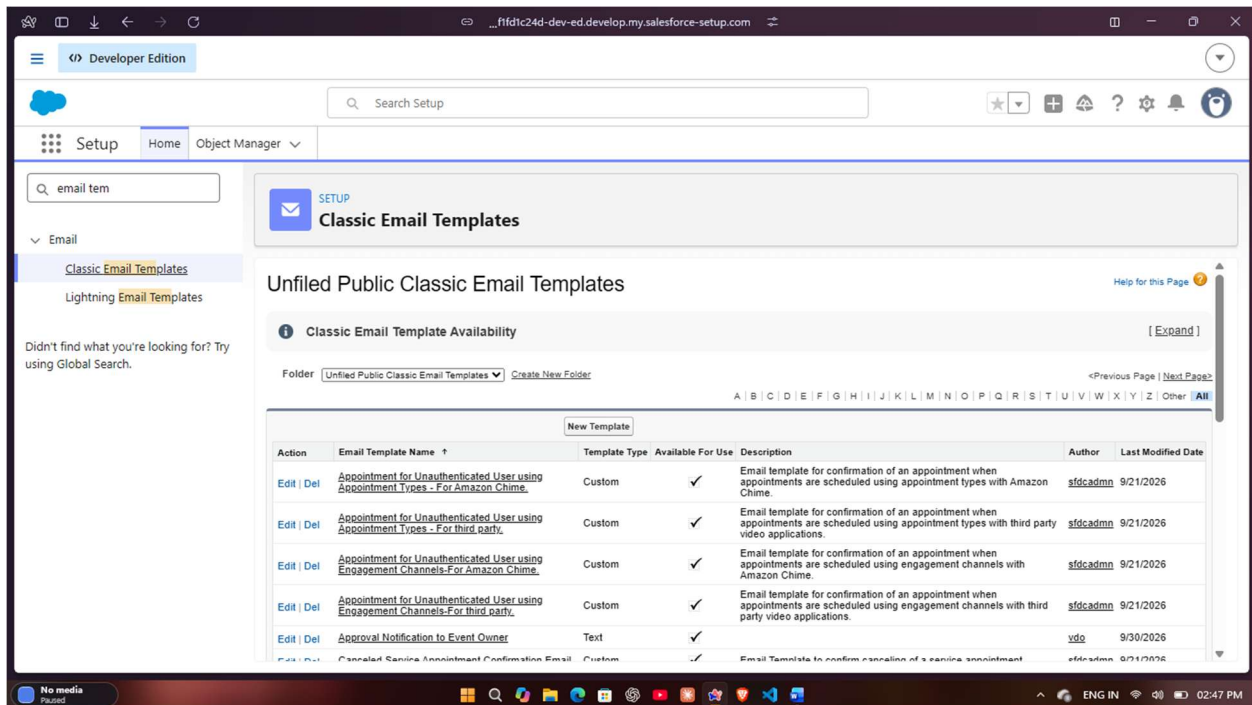


Figure 5: Event cancellation approval process



Milestone 7: Flow – 3-Day Reminder

Scenario: when an Event is created or updated with Event Status = Confirmed, and it has a client with an email address, the client receives a reminder three days before the event date.

Flow Element	Configuration
Flow type	Record-Triggered Flow on Event (created or updated), optimized for Actions and Related Records
Entry condition	Event Status equals Confirmed
Scheduled path	3_Days_Before: time source Event Date, offset 3 days before
Text templates	Subject and Body resources that include the event name, date, venue, and type
Action	Send Email (Alert_Client_3Day_Reminder) to the Client email, with the Event Owner in CC
Flow label	Client Reminder - 3 Days Before (saved and activated)

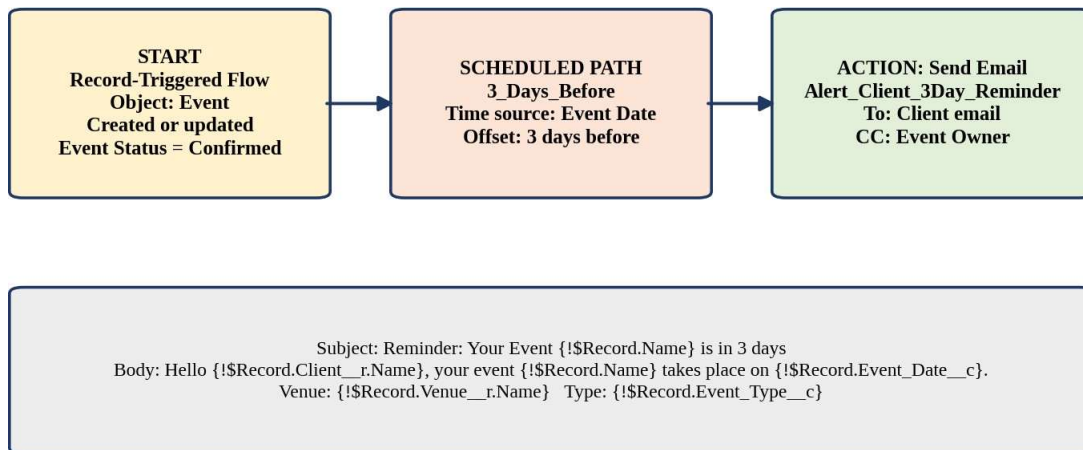
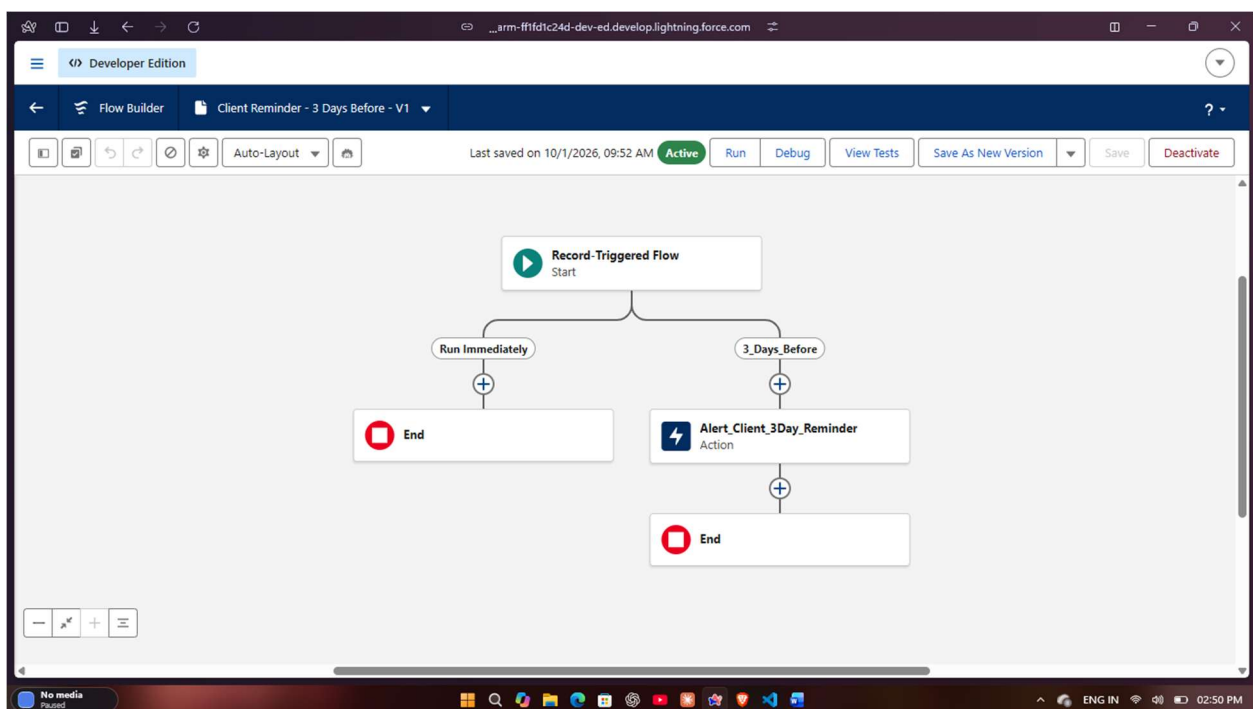


Figure 6: 3-day reminder flow logic



Milestone 8: Apex Class – VenueStatusHelper

Scenario: when an event is confirmed, its venue is marked Reserved; when an event is canceled, the venue is marked Available again. The class is created in the Developer Console and called from the trigger EventTrigger13.

```

public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events) {
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){

```

```

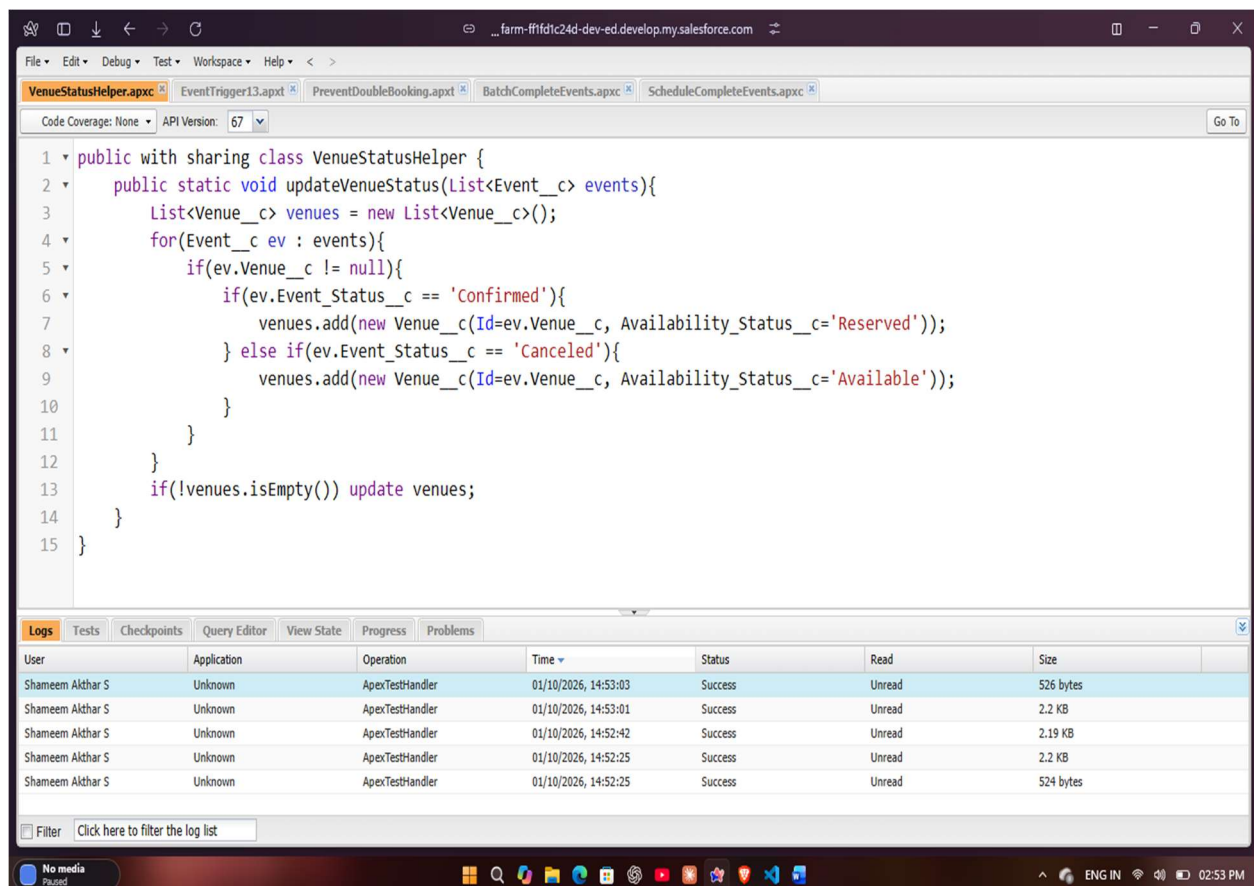
        if(ev.Event_Status__c == 'Confirmed'){
            venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Reserved'));
        } else if(ev.Event_Status__c == 'Canceled'){
            venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Available'));
        }
    }
}
if(!venues.isEmpty()) update venues;
}
}

```

```

trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new);
}

```



Milestone 9: Apex Trigger – PreventDoubleBooking

Scenario: two events must not be booked at the same venue on the same date. The trigger runs before insert and before update, compares the event with existing events at that venue, and shows an error if the date is already taken.

```

trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
}

```

```

for(Event__c ev : Trigger.new){
    if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
}
Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
                  FROM Event__c WHERE Venue__c IN :venueIds]){
    if(!venueEventMap.containsKey(e.Venue__c)){
        venueEventMap.put(e.Venue__c, new List<Event__c>());
    }
    venueEventMap.get(e.Venue__c).add(e);
}
for(Event__c ev : Trigger.new){
    if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
        for(Event__c existing : venueEventMap.get(ev.Venue__c)){
            if(existing.Event_Date__c == ev.Event_Date__c && existing.Id !=
ev.Id){
                ev.addError('This Venue is already booked on this date.');
```

Milestone 10: Batch and Scheduled Apex

Scenario: every night, all events whose date has passed are marked Completed. The batch class does the update and the schedulable class runs it.

```

global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator(
            'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c WHERE
Event_Date__c < TODAY AND Event_Status__c != \'Completed\'
        ');
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){
            ev.Event_Status__c = 'Completed';
        }
        update scope;
    }
    global void finish(Database.BatchableContext bc){
        System.debug('Past events marked as Completed');
    }
}

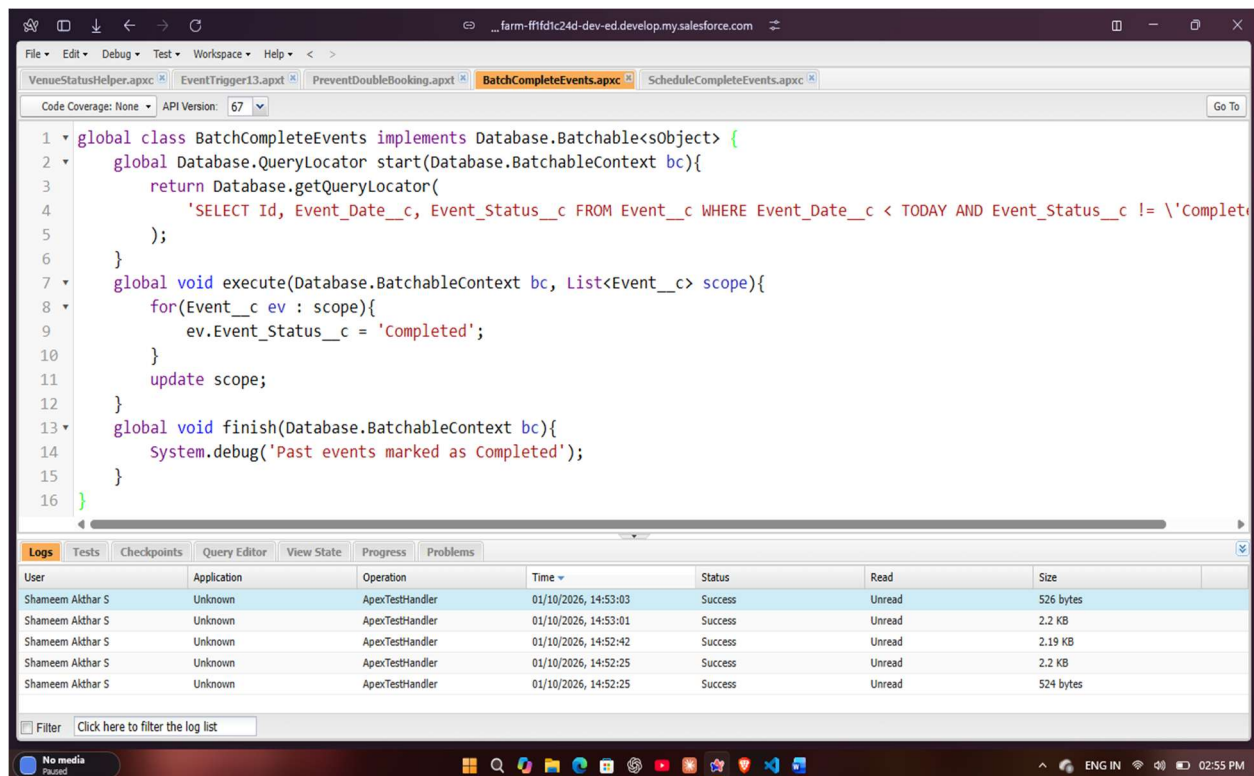
```

```

global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}

```

The job was scheduled from Setup → Apex Classes → Schedule Apex, with the job name Daily Event Completion, the class ScheduleCompleteEvents, a weekly frequency on all days, and a preferred start time of 8:00 PM.

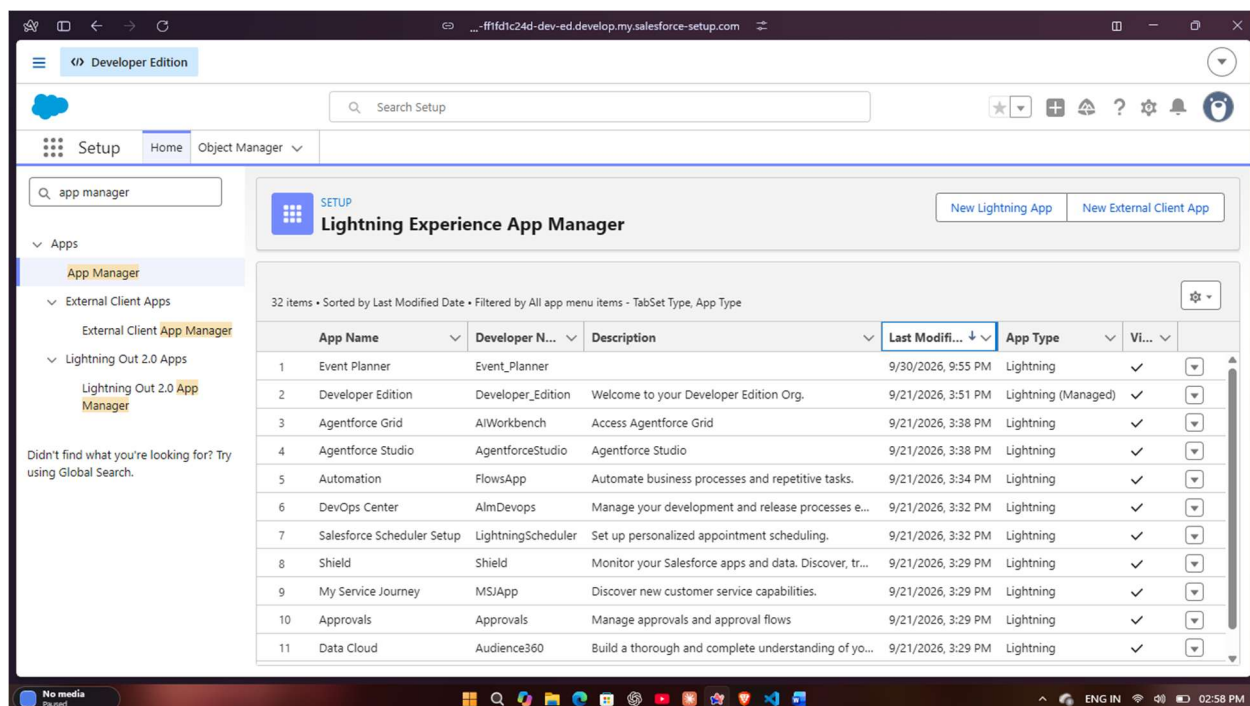


Phase 3: UI/UX Development and Customization

This phase gave users a simple workspace. A custom Lightning App collects Events, Clients, Vendors, Venues, and Feedback in one place, and reports and a dashboard turn the data into visual summaries of schedules and budgets.

Milestone 11: Lightning App

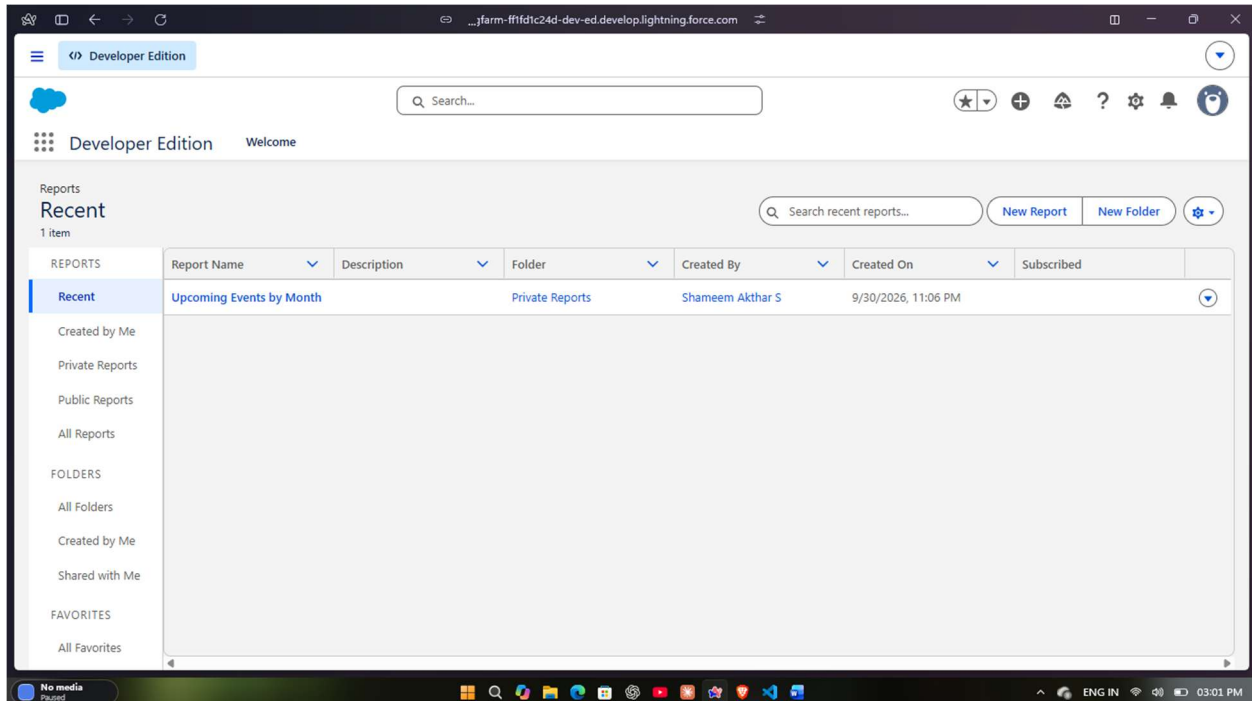
The app Event Planner was created from Setup → App Manager → New Lightning App. Its navigation items are Event, Client, Vendor, Venue, Feedback, Reports, and Dashboards, and it was made available to the System Administrator profile first. The other profiles were given access later through their profile settings.



Milestone 12: Reports

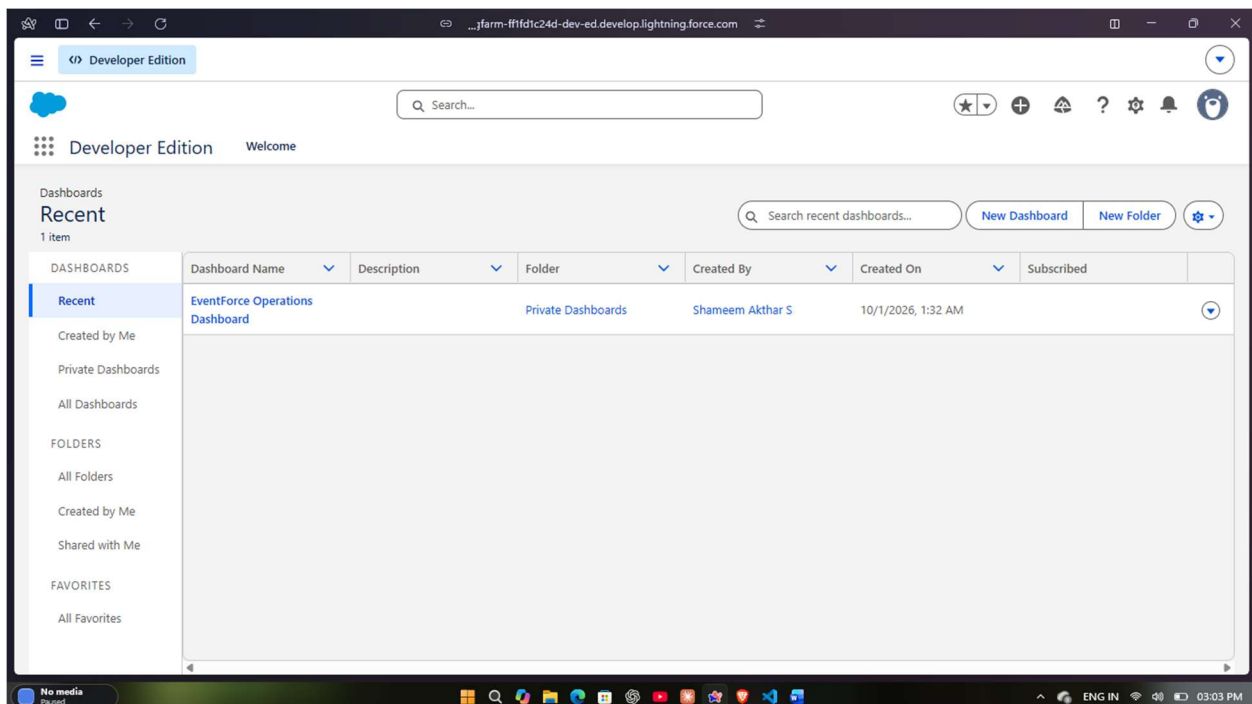
A report folder named EventForce Reports was created and shared with the Event Coordinator and Vendor Manager roles with View access. At least 10 sample records were entered in each object before building the report.

Setting	Report 1: Upcoming Events by Month
Columns	Event Name, Event Date, Event Type, Client Name, Venue Name, Event Budget
Filters	Show Me = All Events; Event Date on or after Today (or next 30 days); Event Status not equal to Canceled
Grouping	Event Date grouped by Calendar Month
Summaries	Row Count and Sum of Event Budget
Chart	Column (or line) chart by month
Saved to	Folder EventForce Reports



Milestone 13: Dashboard

The EventForce Operations Dashboard was created with a widget based on the Upcoming Events by Month report: a donut chart showing up to 6 values, with the subtitle Upcoming Events Overview and the Light theme.



Phase 4: Data Migration, Testing and Security

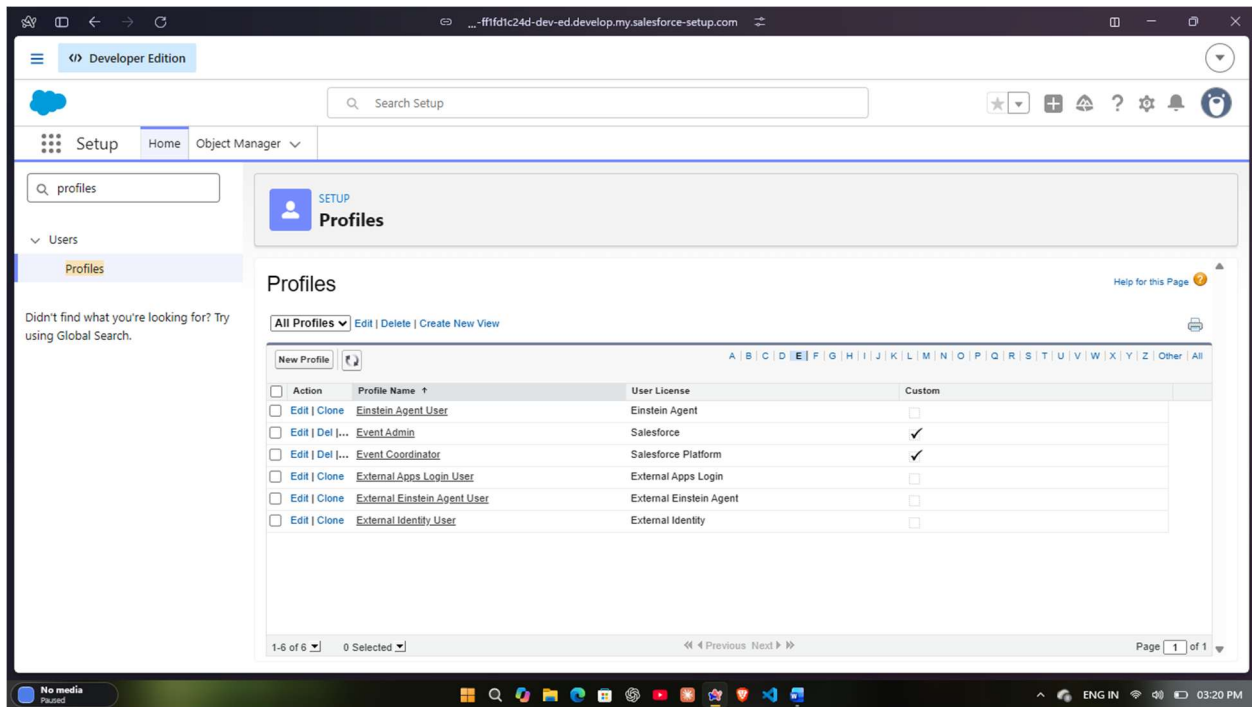
Before deployment, validated records for Events, Clients, Vendors, Venues, and Feedback were loaded with the Data Import Wizard after checking for duplicates. Users were created for the Event Admin, Event Coordinator, Vendor Manager, and Client roles, each with a matching profile and role. Organization-wide defaults were set to Private and a sharing rule opens selected records to vendors. Testing then confirmed data accuracy, automation behavior, and user permissions.

Milestone 14: Profiles

A profile defines what a user can do: object and field permissions, tabs, apps, and login settings. Four custom profiles were created.

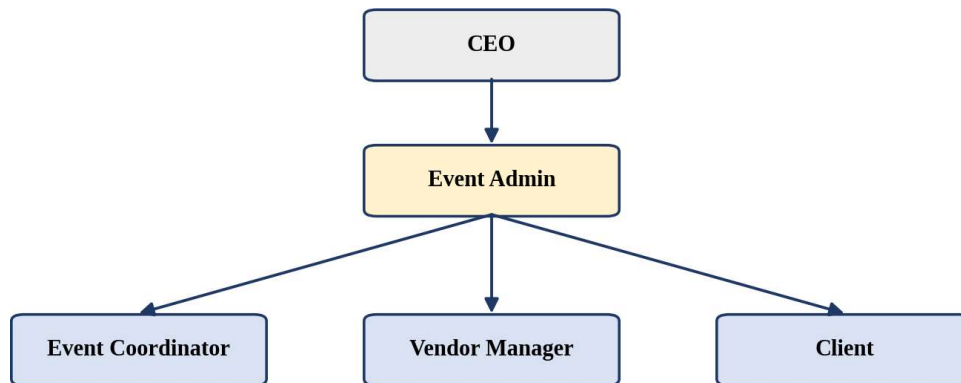
Profile	Cloned From	Object Access
Event Admin	System Administrator	Full access (inherits Modify All Data and CRUD on all objects)
Event Coordinator	Standard Platform User	Event: create, read, edit, delete. Client: read, edit. Vendor and Venue: read. Feedback: create, read
Vendor Manager	Standard Platform User	Vendor: create, read, edit, delete. Event, Venue, Client, Feedback: read only
Client Profile	Standard Platform User	Event: read only (own events). Feedback: create, read. Vendor and Venue: no access

For the three non-admin profiles, Event Planner was set as the default app, the session timeout was set to 2 hours of inactivity, passwords were set to never expire, and the minimum password length was set to 8.



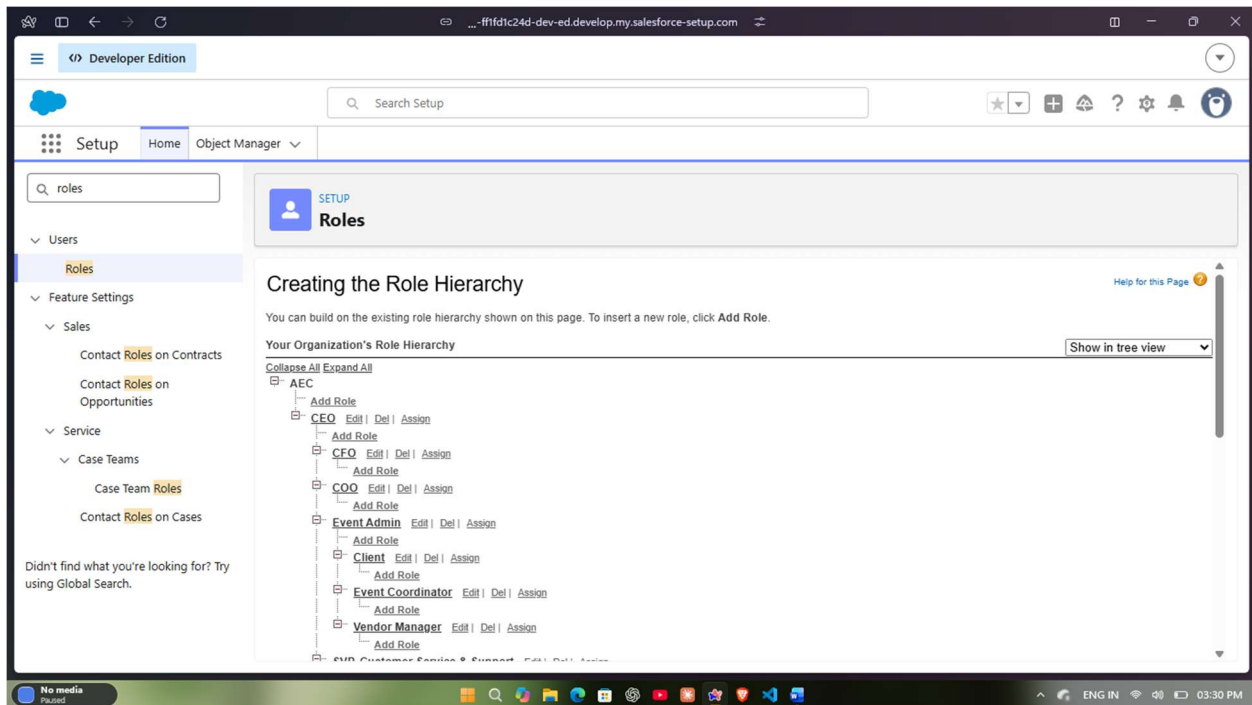
Milestone 15: Roles

Roles control record visibility through a hierarchy. Event Admin was added under the CEO role, and Event Coordinator, Vendor Manager, and Client were created below Event Admin.



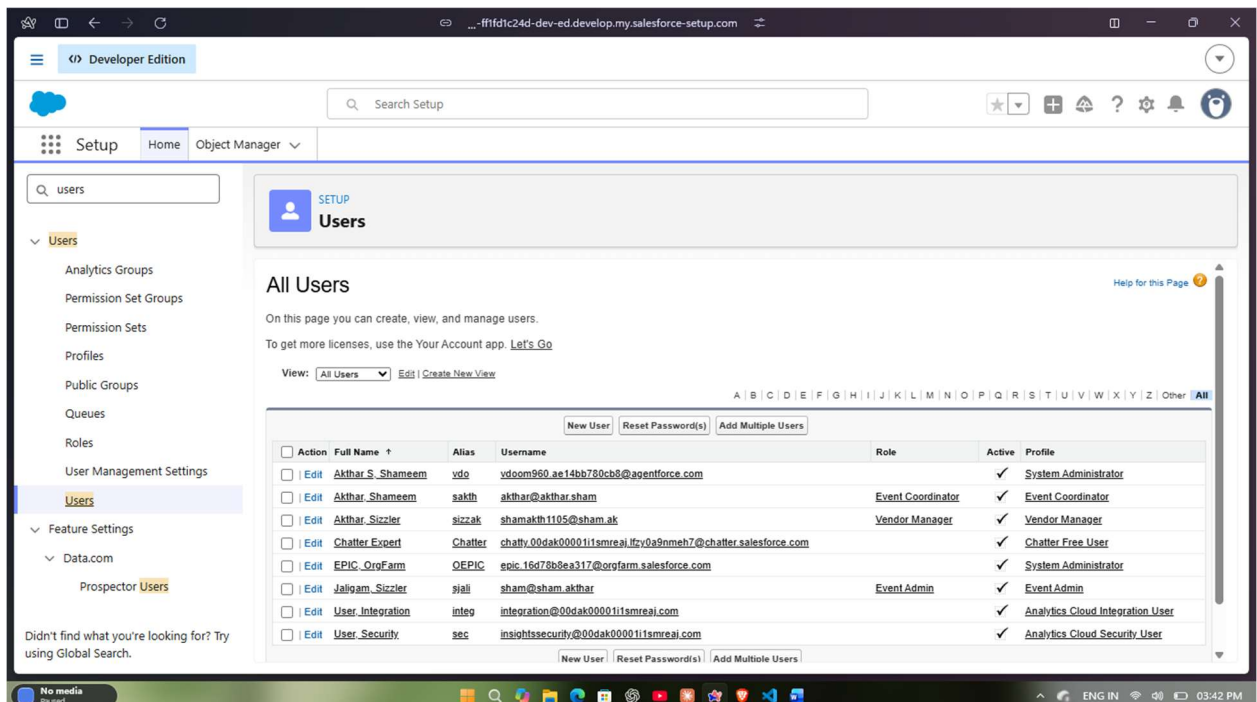
Role hierarchy: users in higher roles can view records owned by users below them.

Figure 7: Role hierarchy of the Event Planner application



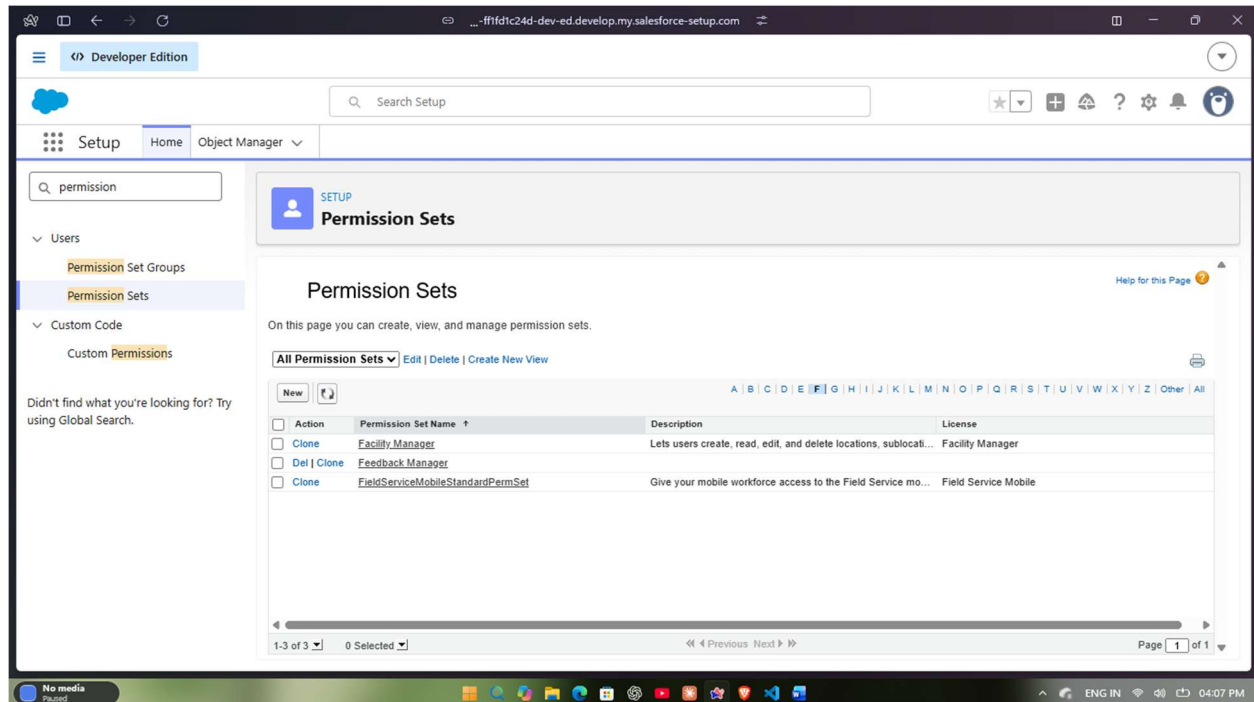
Milestone 16: Users

One user was created for each role from Setup → Users → New User. The Event Admin user has the Salesforce license; the other users have the Salesforce Platform license, each with the profile and role that match. A Developer org allows only a limited number of full Salesforce licenses, so a license may need to be freed or changed if none is available.



Milestone 17: Permission Set

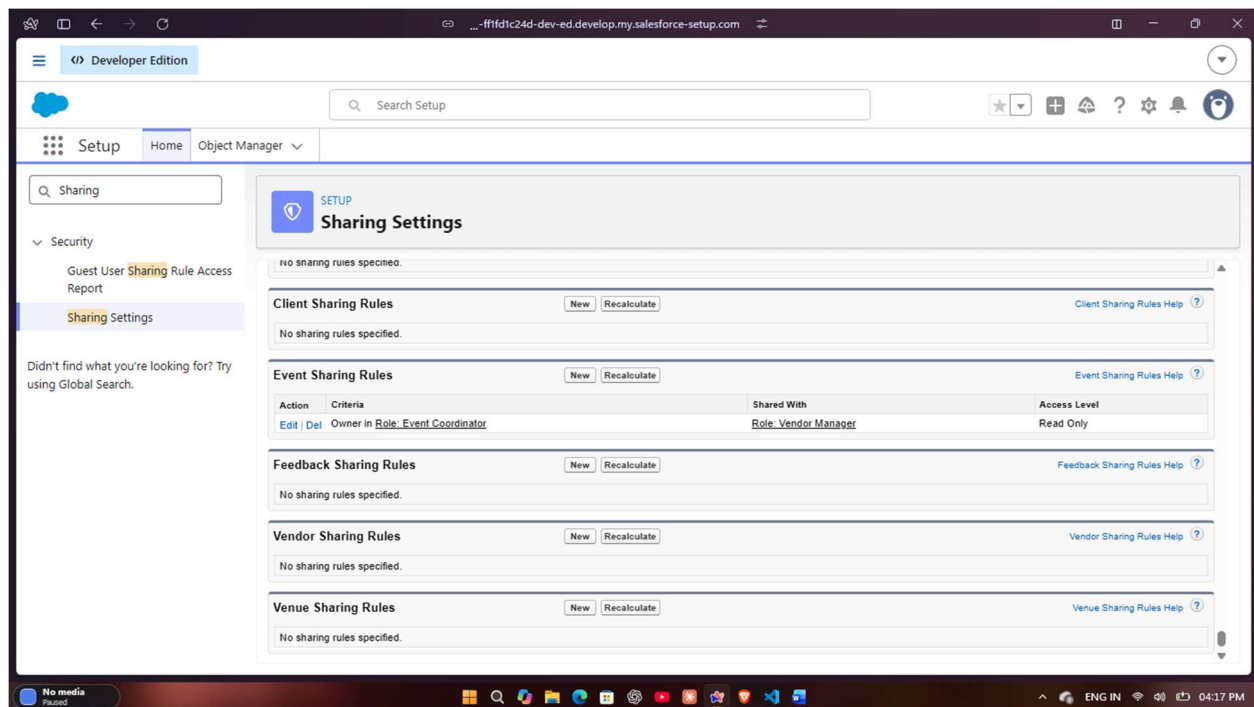
A permission set named Feedback Manager was created to give Read, Create, Edit, and Delete access on the Feedback object, and it was assigned to the Event Coordinator user. This adds access without changing the user's profile.



Milestone 18: Sharing Settings

Organization-wide defaults set the baseline access to records. For the Event object, both default internal access and default external access were set to Private. A criteria-free, role-based sharing rule then extends access where it is needed:

Setting	Value
Rule label	Event_Sharing_For_Vendors
Records to share	Members of role Event Coordinator
Share with	Role Vendor Manager
Access level	Read Only



Phase 5: Deployment, Documentation and Maintenance

Deployment Strategy

All configuration, customization, and automation was built and tested in a Salesforce Developer Edition org. Moving to a live production org was not carried out in this project, because a Developer Edition org is a standalone environment that is not connected to a production instance. In a real implementation the work would be moved as follows:

- Build and test in a sandbox environment.
- Package metadata (objects, fields, flows, Apex, reports, and dashboards) and keep it under version control.
- Deploy to production with change sets, or with Salesforce DX and a CI/CD pipeline such as GitHub.
- Re-run the test cases in production and confirm that the automations and security settings behave the same way.

This phase therefore concentrated on checking that every component is well structured and ready to be migrated when required.

Maintenance, Monitoring and Troubleshooting

Regular maintenance keeps event, client, vendor, and feedback data accurate. Flows, scheduled Apex jobs, and approval processes should be monitored so that failures are caught early. Troubleshooting covers:

- Resolving user access errors and permission conflicts.
- Debugging Apex triggers and flow logic.
- Adjusting validation rules and sharing settings as business needs change.

Documentation Summary

This document records the purpose, data model, configuration, automation, security model, and testing of the EventForce Management System. It maps each business requirement to a Salesforce feature and serves as a blueprint for future developers and administrators, supporting training, maintenance, and knowledge transfer.

7. FUNCTIONAL AND PERFORMANCE TESTING

Each feature was tested in the Developer org, and screenshots were captured as evidence. Enter the result of each test and attach the screenshot.

ID	Feature Tested	Test Step	Expected Result	Result
T-01	Event creation	Create an Event with type, date, and status	Record saved; Event Budget is calculated from the type	
T-02	Email validation	Enter an invalid email on a Client	Error "Please Enter Valid Email Address"	
T-03	Double booking	Create two events at one venue on the same date	Error "This Venue is already booked on this date."	
T-04	Venue status	Confirm an event, then cancel it	Venue becomes Reserved, then Available	
T-05	Cancellation approval	Set status to Pending Cancellation and approve	Approval request raised; status becomes Canceled; emails sent	
T-06	Cancellation rejection	Reject a cancellation request	Status becomes Rejected; owner is notified	
T-07	3-day reminder	Confirm an event dated 3 days ahead	Client receives the reminder email	
T-08	Batch job	Run the batch for events with past dates	Those events are set to Completed	
T-09	Security	Log in as each role	Each user sees only the permitted objects and records	
T-10	Reports and dashboard	Open Upcoming Events by Month and the dashboard	Data, grouping, and charts display correctly	

8. ADVANTAGES AND DISADVANTAGES

Advantages

- **Centralized data:** events, clients, vendors, venues, and feedback live in one platform.
- **Process automation:** flows and batch jobs reduce manual effort.
- **No double bookings:** the trigger and helper class keep venue schedules accurate.
- **Controlled cancellations:** the approval process gives a structured path with notifications.
- **Operational visibility:** reports and the dashboard summarize schedules and budgets.
- **Secure access:** profiles, roles, permission sets, OWD, and sharing rules limit what each user sees.
- **Scalability:** the platform can grow with new requirements.
- **Data quality:** validation rules and pre-checked imports keep data consistent.

Disadvantages

- **Developer Edition limits:** the project was built for academic purposes and is not connected to a production org.
- **No production deployment:** migration to a live environment was outside the scope.
- **Extra tools needed:** real deployment needs sandboxes, change sets, Salesforce DX, or CI/CD tools.
- **Configuration effort:** flows, Apex, approvals, and security need Salesforce knowledge to manage.
- **Ongoing maintenance:** automations and sharing settings need regular monitoring.

9. CONCLUSION

The EventForce Management System shows how Salesforce can streamline and automate event planning. Events, clients, vendors, venues, feedback, and vendor assignments are now managed on one platform, with automation for reminders, cancellation approvals, venue availability, double-booking prevention, and event completion.

Building the project taught the full implementation cycle: designing a data model, writing validation rules, flows, and Apex, securing data with profiles, roles, permission sets, and sharing rules, migrating data, testing, and documenting the result. Although the solution was built and tested in a Developer Edition org without a production deployment, it follows real-world Salesforce practice and provides a base for future growth.

10. FUTURE SCOPE

- Production deployment using sandboxes, change sets, or Salesforce DX.
- More event automation for planning and operational tasks.
- Advanced vendor tracking and performance management.

- More reports and dashboards on events, venues, vendors, and feedback.
- Integration with external systems used in event operations.
- Mobile access for managing events on the move.
- Scaling the system for larger numbers of events, clients, vendors, venues, and users.
- Continuous improvement of flows, Apex, validation rules, and sharing settings.

11. APPENDIX

- **A. Custom Objects:** Event, Client, Vendor, Venue, EventVendor, Feedback.
- **B. Automation:** 3-day reminder flow, validation rule, Event_Cancellation_Process approval process, Apex triggers and classes.
- **C. Processing:** BatchCompleteEvents and ScheduleCompleteEvents (Daily Event Completion job).
- **D. Security:** Event Admin, Event Coordinator, Vendor Manager, and Client profiles; role hierarchy; Feedback Manager permission set; OWD and Event_Sharing_For_Vendors.
- **E. Analytics:** Upcoming Events by Month report and EventForce Operations Dashboard.++